

Game Power Plan Switcher

Full guide for users and developers

Version 2.3.6 | 6 October 2026 | NN6

Game Power Plan Switcher automatically selects an installed Windows power plan while a watched game is running, then restores Default when the last eligible process closes. Version 2.3.6 gives the former NN6 Power Plan project a descriptive public name. The native Rust engine, C++ Windows bridge, Slint interface and saved settings remain compatible.

Use this guide to configure the app, interpret telemetry, troubleshoot local behavior, and build or review the source. Default behavior is any matching process selects Gaming; the last eligible process closing selects Default. Optional battery, time, foreground and manual policies can override that decision.

Contents

Page	Topic
2	First launch and power plan setup
3	Game profiles and process tuning
4	Automation and plan tools
5	Overview and CPU topology
6	Hardware sensors and the telemetry HUD
7	Window controls tray and startup
8	Engine architecture and recovery
9	Source map and build instructions
10	Local data and diagnostics
11	Licensing security and release provenance
12	Troubleshooting and verification

Supported environment

The delivered executable targets Windows x64 with an OpenGL-capable graphics driver. The project uses Rust edition 2024, C++23 and pinned Slint 1.18.1 with the FemtoVG OpenGL renderer. The tested development environment is Windows 11. ARM64, every CPU generation and every multi-monitor DPI combination are not certified by this build.

Monitoring runs under the current user without elevation. Readings and power operations depend on Windows, drivers and provider permissions. This guide describes implemented behavior; the release verification report records the exact build and conditions actually tested.

First launch and power plan setup

Choose the installed plans

Open Game-Power-Plan-Switcher-2.3.6.exe. In Settings, select Gaming and Default from the installed plan list. The selectors open a searchable picker with keyboard navigation and an explicit empty state. Use different plans for the two roles; an invalid selection is rejected before the active configuration is replaced.

```
powercfg /list
powercfg /getactivescheme
```

The original machine used LowLatency-Intel v2 as Gaming and Balanced as Default. These identifiers are useful for that installation; another computer may require different installed plans.

Role	Original GUID
Gaming	319f9863-5b07-4924-b71a-3a687a155782
Default	381b4222-f694-41f0-9685-ff5bb260df2e

Transfer an existing monitor deliberately

If Overview says that the previous monitor is running, the older monitor still owns the shared lock. Use this app instead requests handover of the exact owned GamePowerPlan-RainbowSix task for the current user. It does not disable unrelated scheduled tasks. A pending indicator lasts until the engine acknowledges the outcome; failure keeps the banner actionable.

Updating the desktop shortcut does not replace an older GUI already running. Exit that app normally when ready, then open the latest executable. Participating GUI versions share singleton behavior for the same user and Windows session.

Confirm the first automatic cycle

1. Review Workspace aliases and Settings policies, then start monitoring.
2. Launch one configured game. Check the current plan in the app and with `powercfg /getactivescheme`.
3. Close the game normally. If another watched process remains, Gaming can remain active.
4. After the final eligible process exits, confirm Default. Inspect the timeline and switch log if the result differs.
5. Pause monitoring to restore Default and any tracked overlay or process scheduling changes.

Do not force-close a real game to test recovery. Development integration tests use disposable processes and isolated data. Detection latency depends on the available Windows event backend; an ordinary-token fallback can take about a second to notice a start.

Game profiles and process tuning

Find the process that actually runs

Launch the game and inspect Task Manager under Details. Match the executable name rather than the launcher name. The app does not require Steam or a Steam App ID for runtime detection. The Running picker offers read-only process candidates; Foreground app uses the last external foreground application captured by the app.

```
Get-Process -Name '*Rainbow*' |  
    Select-Object ProcessName, Id, Path
```

Rainbow Six Siege is configured with RainbowSix.exe, RainbowSixSiege.exe and RainbowSix_DX11.exe. Any configured alias can make the profile eligible. Process names are compared without case sensitivity.

Add and manage profiles

- Browse reads an executable icon and metadata without launching it.
- Bulk input accepts comma, semicolon or newline separators. It removes outer quotes and path prefixes, validates executable names, and ignores duplicates without case sensitivity.
- Accepted names use letters, digits, underscore or hyphen followed by .exe. Invalid input receives inline guidance and cannot be added until corrected.
- Discover reads accessible launcher library metadata. Review candidates because helpers and protected libraries may need manual correction. Auto-discover is opt-in.
- Saved profile search is literal. Remove and Profile settings require a selected profile. Up to 500 executable names are supported.

Per game policy

Profile settings can select an installed per-game power plan instead of the global Gaming plan. If several eligible profiles run, stable saved-profile order decides which profile wins. A Default-plan override is allowed for a particular profile.

Affinity and priority are optional. Blank affinity leaves scheduling inherited; named affinity policies include performance, efficiency, physical and performance-physical. Physical policies choose one logical thread per physical core within a supported single processor group. They do not disable SMT in firmware. Multi-group or out-of-mask requests are refused.

Reset to Default changes only the affinity and priority draft. Save profile submits the transaction. Failed, timed-out or superseded commands preserve the draft so a background receipt cannot erase newer edits. Protected processes can reject scheduling changes; review the reported error instead of assuming success.

Automation and plan tools

Policy order

The engine evaluates the following priorities in order. All actual plan writes stay in the serialized engine path, with current-plan checks and recovery bookkeeping.

Priority	Decision
1	Enabled battery guard selects Default on battery power or when the power source is unknown.
2	An active manual Gaming or Default choice applies next, unless blocked by the battery guard.
3	An enabled local-time window gates automatic game eligibility. Outside it, Default is selected.
4	The first configured eligible running profile selects its override or the global Gaming plan.
5	With no eligible profile, the configured Default plan is selected.

Configure the rules

Require a foreground game is optional and off by default. With it enabled, a watched process also needs foreground focus. Trigger by Time of Day uses local HH:MM values, supports overnight intervals and treats the start as inclusive and end as exclusive. It does not activate Gaming merely because a time window begins.

New configurations enable the battery guard. Migrated configurations preserve their earlier behavior, so inspect the saved setting. Battery and time observations run on a nominal five-second policy tick only when the relevant rules or retry state need them. A failed application can be retried without requiring an AC/DC change.

Editing a plan or policy while monitoring uses one engine transaction: validate installed targets, pause and restore prior changes, save, then resume if previously running. The GUI shows Applying until the matching receipt arrives. Failed restoration prevents committing a new configuration; rollback errors remain visible.

Manage power plans explicitly

Power tools export a selected plan as .pow, import a .pow into a fresh GUID, duplicate a plan, or open advanced Windows settings. These commands are separate from monitoring. Importing or duplicating a plan does not request activation.

Some .pow operations require privileges Windows does not grant an ordinary token. When that happens, Retry with Windows approval explains and retries the exact retained operation through the native approval prompt. Elevation is explicit; the monitoring process remains unelevated. Keep an uncertain import or duplicate GUID from the error report before retrying, so you do not lose track of a possible created plan.

Overview and CPU topology

Interpret the live display

Current Plan reports the engine-observed active plan alongside monitoring status. The CPU graph shows up to 30 seconds of measured busy-time samples with plan-transition markers. Hover reveals the sample time, CPU value and plan context. Gaps represent missing samples rather than fabricated zero values.

CPU utilization uses Windows PDH Processor Information counters, nominally once per second while a view needs them. Pausing power automation does not freeze the CPU display. A value of 0% is measured idle activity; unavailable or warming up is a different state. These samples do not identify the process currently executing on each core.

How the sections are chosen

Windows topology	Presentation
Intel with distinct efficiency classes	P-CORES and E-CORES sections, with physical-core and logical-thread counts calculated independently.
Homogeneous or unclassified CPU	Neutral logical-processor section. A zero efficiency class alone does not imply E-core.
AMD with valid shared L3 masks	CACHE GROUP sections report the cores and threads sharing each L3 domain and its size.
Incomplete relationships	Known data remains visible and unmatched processors stay neutral.

Windows GetLogicalProcessorInformationEx supplies the relationships. CUID supplies vendor and model identity only. L3 sharing is not guaranteed physical CCD or CCX identity; cache-size asymmetry is not proof of 3D V-Cache. Lower Intel classes are not labeled LP-E without a verified identification method.

The chip grid and heatmap scroll through all reported processors. Compact uses a smaller chart and eight heatmap columns; Expanded offers a taller chart and sixteen columns. Inapplicable sections are omitted without reserving empty rows. Info shows the model, counts and detector details.

Session time and the timeline

Gaming and Default counters cover the connected observations in this GUI session. Unknown plans and disconnected intervals are excluded. Repeated exports cannot count disconnected time from a cached plan; accounting resumes with the next live observation. The counters are not persistent lifetime totals or game runtime. The timeline retains up to 64 transitions with source and target plans, cause, available process/PID, account and API duration. API duration is not total process detection latency.

Connect Sensor explains why energy savings needs a meter and baseline. Temperature or package power alone does not establish a savings percentage.

Hardware sensors and the telemetry HUD

Available channels

Source	Metrics and cadence
NVIDIA NVML	Supported GPU temperature, utilization, fan percentage and actual RPM; nominal two-second direct sampling while a visible view needs it.
AMD ADL8 PMLog	Supported AMD GPU metrics through an owned read session; availability depends on adapter and driver support.
LHM or OHM WMI	CPU temperature, package power, board and fallback sensors from an already running provider; independent ten-second worker with thirty-second local cache expiry.

Each measurement has independent availability. Fan percentage is not converted into invented RPM. A reported zero remains valid. The app identifies the selected accessible GPU; it does not claim that this is the adapter used by a particular game. Partially supported direct readings are not filled using another GPU.

Connect an existing provider

1. Confirm that the installed GPU driver or the separate hardware-monitor application can read the desired sensor.
2. For CPU and board metrics, keep a compatible LibreHardwareMonitor or OpenHardwareMonitor WMI provider running. Follow its own hardware and permission instructions.
3. Open Settings and Integrations, then select Connect sensors or Retry. Read the provider status if values remain unavailable.
4. Keep unavailable values as diagnostic evidence. The app does not install a driver or silently download a service to fill missing metrics.

The WMI receipt age is local observation time, not a hardware acquisition timestamp. An unchanged number proves neither freshness nor staleness. Suspending both views invalidates cached fallback observations; resuming requests fresh values. WMI work does not block the direct GPU sampling loop.

Desktop telemetry HUD

The optional HUD is a topmost desktop window with selectable corner, opacity and a separate shortcut. It shows available measurements and unavailable states without injecting into a game. Applying its settings saves the preference. Disabling the HUD drops the window/component and cancels pending creation; re-enabling creates it again after native-window readiness.

The Best performance overlay option is a Windows power-mode overlay, not this visual HUD. Its previous state is recorded and restored. The OS exports used for that feature are not a guaranteed public SDK contract, so refusal is reported and the option remains off by default.

Window controls tray and startup

Choose a comfortable layout

New preferences use Compact at 584 by 526 logical pixels. Expanded uses 800 by 600. Existing saved preferences remain intact. Light, dark and Follow Windows theme are available, along with Reduce interface motion and an optional native Windows frame. Simulated glass uses local gradients; it does not blur other applications.

Action	Behavior
Main yellow traffic light	Minimizes the main window; background monitoring continues.
Main red traffic light	Hides to tray when configured, otherwise performs normal exit.
Maintainer or About red light	Closes only the dialog. These dialogs have only the red dismiss control.
Ctrl and Enter inside the app	Toggles monitoring when no modal dialog is open.
Ctrl Alt P by default	Registered global shortcut for manual Gaming/Default selection.
Tray Exit	Exits and requests restoration through the engine.

Global shortcuts and tray

Apply validates and registers a shortcut with Windows, reporting conflicts rather than displaying success prematurely. A blank chord disables registration. The plan and HUD shortcuts must differ. Manual plan intent clears on pause/resume, settings/import changes or the next watched lifecycle event; the battery guard remains higher priority.

Repeated launches send SHOW to the existing GUI for the same user and Windows session. A rate-limited notice can be queued through its existing tray icon. Windows may suppress notifications; foreground restoration does not depend on a balloon appearing. Distinct user sessions remain separate.

Start at user logon

Startup keeps the compatibility filename NN6-PowerPlan-Native.exe in the per-user NativeApp directory and sets HKCU\Software\Microsoft\Windows\CurrentVersion\Run with value NN6PowerPlanNative. The same control removes that registration. Start minimized controls the initial window state. Startup runs after logon, not before it, and has no watchdog that reopens a deliberately closed GUI.

To stop using the app, remove its Startup registration, pause monitoring, and choose Exit. Keep settings and recovery files until normal restoration finishes. Removing old project builds is different from uninstalling the per-user startup copy or the historical scheduled monitor.

Engine architecture and recovery

Three process roles

Role	Responsibility
GUI	Slint views, user commands, presentation models, tray interaction and diagnostics.
Engine daemon	Serialized policy decisions, process subscriptions, power writes, scheduling changes and restoration journal.
Guardian	Restoration support if the engine crashes while changes are active.

The GUI singleton is acquired before Slint initialization. Its Global named mutex is scoped by user SID, Windows session and isolated preview context. Duplicates use a bounded, user-only, local named-pipe SHOW exchange and exit without starting a second renderer or engine. Failed handoff does not authorize a second GUI.

Detection and plan changes

The native bridge subscribes to process start and stop events and foreground window changes. Where ordinary-token Win32_ProcessStartTrace access is denied, authenticated WMI subscriptions fall back to intrinsic WITHIN 1 events. Known process exits also use direct process-handle waits. Initial process enumeration is not a repeated five-second detection loop.

PowerGetActiveScheme is read before PowerSetActiveScheme. The engine avoids a redundant write when the target is already active and verifies the result after a mutation. Normal monitoring activates installed schemes without rewriting their definitions. A shared monitor lock prevents competing owners.

Before applying reversible overlay or scheduling changes, the engine records restoration state. Pause and Exit restore Default and tracked changes. A failed restoration stays journaled and blocks a new monitoring transaction until handled. The guardian and subsequent startup can retry recovery; they are not defeated by a UI integrity gate.

IPC and interface work

Overlapped named pipes carry commands and receipts. A bounded per-user memory-mapped ring carries encoded telemetry snapshots. Protocol Buffers is binary serialization with copying; it is not claimed to be zero-copy. Command IDs correlate each client request with success or failure, and legacy zero-ID messages cannot erase a tracked receipt.

Stable Slint models update changed rows rather than rebuilding all vectors. Comparisons include freshness and availability, so unchanged load does not conceal stale data. Hidden and monitored-fullscreen views suppress model, chart, sensor and animation work; the engine remains active. Fullscreen bounds are a heuristic, not proof of exclusive DirectX mode. The main graphics surface may remain cached.

Source map and build instructions

Download source from github.com/112-stack/game-power-plan-switcher or extract the source archive attached to the matching release. Run Build.ps1 from that source folder. Keep Cargo.lock and pinned Slint when reproducing the supported toolchain.

Files	Responsibility
ui/app.slint	Reusable controls, views, modals and AppWindow properties/callbacks.
src/ui.rs and ui_pro.rs	UI orchestration, integration callbacks, view suspension and HUD lifecycle.
src/ui_cpu.rs and cpu_topology.rs	Stable CPU presentation models and checked Windows topology parsing.
src/ui_requests.rs and pro_view.rs	Correlated draft transactions and observed charts/session data.
src/engine.rs and power.rs	Serialized engine state, power orchestration and restoration.
src/gui_instance.rs and ipc.rs	GUI ownership, SHOW protocol, pipes and mapped telemetry.
src/sensors.rs and system_integration.rs	Direct sensors, provider workers and native integration bindings.
native/bridge.cpp and pro_bridge.cpp	C++23 Windows event, power, scheduling and shell interfaces.
build.rs and native/app.rc	Slint compilation, native build, icon resources and build provenance.

Supporting modules separate policy decisions (automation.rs), launcher metadata (discovery.rs), records (model.rs), appearance (preferences.rs), CSV/JSON output (session_export.rs) and release comparison (protection.rs). runtime_scope.rs defines production, preview and test data/IPC identity. Diagnostic helpers remain separate from the normal launch path.

Build the portable source

Use Rust 1.99.0 for x86_64-pc-windows-gnullvm and a compatible LLVM-MinGW UCRT toolchain. The script validates clang, clang++, windres and llvm-ar and sets the linker consistently. Do not mix the GNU Rust host with the UCRT LLVM toolchain. Replace the example toolchain path with its actual local directory.

```
.\Build.ps1 -LlvmMingw 'C:\Tools\llvm-mingw' -Action test
.\Build.ps1 -LlvmMingw 'C:\Tools\llvm-mingw' -Action build
```

Build.ps1 uses Cargo --locked and release mode unless DebugBuild is requested. Its executable is target\x86_64-pc-windows-gnullvm\release\Game-Power-Plan-Switcher.exe. The workspace wrapper work/Build-Native.ps1 uses the staged toolchain and accepts a versioned OutputPath.

Keep the public AppWindow and protobuf contracts compatible. Preserve Windows header order in C++ files. Prefer a small justified repair over restructuring working recovery, IPC or sensor lifecycles merely for appearance.

Local data and diagnostics

Path relative to the user data directory	Purpose
NN6PowerPlan\native-settings.json	Game aliases, plan selection and engine policies.
NN6PowerPlan\appearance.json	Window, theme, tray, hotkeys and HUD preferences.
NN6PowerPlan\native.log	Operational status and errors; a rotated native.log.1 may also exist.
NN6PowerPlan\recovery.json	Pending restoration state.
NN6PowerPlan\native-crash.txt	Recorded panic details when available.
NN6PowerPlan\native-startup-error.txt	Startup or diagnostic command failure.
NN6PowerPlan\NativeApp\	Stable executable copy used by native startup.
GamePowerPlan\switch.log	Timestamped power-switch records shared with the earlier workflow.

Paths above are relative to %LOCALAPPDATA%. Version 2.3.6 retains them for compatibility. Export dialogs let you choose a destination. Back up configuration after normal Exit; retain an unresolved recovery journal until restoration succeeds.

Export a useful report

Session export includes observed durations and recorded transitions. Diagnostic JSON also includes current status/error, detector and automation state, active plan, history and a bounded 64 KiB tail of native.log. Missing or truncated logs are explicit. Failures outside the retained tail are not recovered by the export.

Before sharing, review account names, executable paths, plan names and process metadata. The app has no phone-home telemetry or online activation. Clicking Discord or Twitch intentionally opens an external application or website.

Read only topology report

From the release folder, choose a writable absolute output path. This command reports CPU relationships without opening a GUI/engine or changing power plans.

```
$report = Join-Path $PWD 'game-power-plan-topology.json'
Start-Process '.\Game-Power-Plan-Switcher-2.3.6.exe' `
  -ArgumentList @('--topology-probe', ('"' + $report + '"')) `
  -WindowState Hidden -Wait
```

Preview isolation

A bare --dry-run launch uses %LOCALAPPDATA%\NN6PowerPlanPreview with separate preview IPC and settings. Explicit NN6_TEST_DATA_DIR paths are normalized and hashed into test IPC identity; dry and live modes use separate channels. The same deliberate data root still has engine-lock exclusion. Initial snapshot mode is checked before HELLO or any queued commands are sent.

For disposable checks, use --dry-run --start-paused with a fresh absolute NN6_TEST_DATA_DIR outside production data and preserve harness cleanup guards. Simulated writes do not prove real switching or restoration.

Licensing security and release provenance

License and attribution

Game Power Plan Switcher is distributed under GPL-3.0-or-later, with Slint using its offered GPLv3 alternative for the combined distribution. Consult LICENSE.txt, NOTICE.txt and THIRD-PARTY-NOTICES.txt for exact terms and separate dependency or asset rights. Earlier MIT grants remain applicable to the material they covered. CITATION.cff provides optional citation metadata.

The release has no activation, hardware lock, debugger or VM detection, falsified readings, punitive cooldown, packer or covert watermark. Trademark guidance does not create a blanket prohibition on GPL-compliant modification or rebranding. This guide does not replace the license text.

Verify a downloaded release

The shipped executable is unsigned unless a publisher separately signs it. First use a trusted external tool to compare its digest with an independently obtained reference; do not launch an untrusted file merely to ask it whether it is trustworthy.

```
$exe = '.\Game-Power-Plan-Switcher-2.3.6.exe'
Get-FileHash $exe -Algorithm SHA256
Get-AuthenticodeSignature $exe
```

Explicit `--write-release-manifest` and `--verify-release` commands compare the final on-disk executable, raw `.text/.rdata` digests, PE flags and build provenance against an external manifest. A replaceable manifest does not authenticate the publisher. Verification does not scan process memory or block startup and recovery automatically.

Release procedure

1. Run the affected checks and full regression suite. Record the exact source, compiler, build inputs and evidence for this revision.
2. Build the final executable, inspect resource versions and all icon frames, and record its signature status.
3. If signing is authorized and a real certificate is available, sign now. Signing changes bytes; regenerate manifests and checksums afterward.
4. Package matching source, dependency notices, verification and documents. Verify the archive content against the canonical source.
5. Publish only to an authorized destination. Optional detached signatures or provenance are separate publisher actions.

RELEASE-SECURITY.md explains signing and provenance. The project is at github.com/112-stack/game-power-plan-switcher. Follow SECURITY.md for vulnerability reports; the retained developer contact is <https://discord.com/users/176078095957098497>. Include version, hash, Windows version and reproduction steps; omit credentials and redact personal logs. No response-time commitment or security certification is implied.

Troubleshooting and verification

Symptom	Check and response
Gaming is not selected	Check the actual executable alias, monitoring ownership, battery guard, time window and foreground requirement. Review the current status and native.log.
Default remains active on AC	If the power source is unknown, an enabled battery guard deliberately keeps Default. Inspect the reported source and selected installed GUIDs.
Previous monitor banner persists	Wait for the command result. Handover denial in dry-run is expected. Do not manually disable unrelated tasks.
Temperature is unavailable	Confirm driver/API support or the running LHM/OHM WMI provider. Retry local sources; missing values do not prevent process monitoring.
Second launch shows an older app	Exit the older participating GUI normally, then relaunch the latest executable. A shortcut update does not replace a process.
Plan export needs approval	Use the explicit Windows approval retry if intended. Record an uncertain newly created GUID before repeating import/duplicate.
UI or graph pauses	Restore the main view or leave monitored fullscreen. Fresh sampling warms up on resume; data is not backfilled with guessed values.
A shortcut is unavailable	Apply the chord and read the Windows registration result. Choose a different chord or clear it to disable the binding.

What to verify for each release

Use the verification report attached to version 2.3.6 for its actual test results. The 124 passing tests, nine singleton checks and seven integrity checks recorded for 2.3.5 are historical evidence for that binary. They are not automatically results for the renamed build. The rename preserves the monitoring architecture and compatibility identifiers; it adds no new performance or hardware certification.

Fixture coverage can test homogeneous, hybrid, shared-L3, asymmetric-cache and processor-group cases without certifying physical AMD hardware. Render-callback counts do not measure GPU memory allocation. A successful dry-run does not prove live power writes, battery transitions, UAC approval or guardian crash restoration.

Reference files

README.md and README.txt explain operation and building. CHANGELOG.md records release changes. IMPLEMENTATION.txt describes the topology and UI design decisions. RELEASE-VERIFICATION.txt identifies current evidence. Cargo.lock and DEPENDENCY-SOURCES.json identify dependencies; the cached dependency archive may be incomplete, so check its index before attempting an offline build.

This guide accompanies the current source and executable. It does not claim guaranteed sub-millisecond detection, fixed working-set limits, zero GPU allocation, zero-copy protobuf or full WCAG certification. Keep test conditions attached to performance and compatibility results.